AgriGantry: how it works

AF-434 wiring + AgriGantry commands | Arduino Mega 2560

The quick explanation

Your computer controls the gantry with hold-to-move keys or typed step commands. The Arduino turns those requests into electrical pulses on the same pins as the working AF-434 program. Motors 1 and 2 move both sides of X together; motor 3 moves Y; motor 4 moves Z. Hold W/S, A/D, or Q/E to move, then release the key to slow smoothly to a stop. All three axes can jog together. Press O to stop every motor immediately without Enter; that stop stays latched until C in keyboard mode or CLEAR in command mode. There are no limit switches, so startup produces no automatic motion or homing.

The wiring it actually uses

Motor / axis	PUL	DIR	ENA	ALM	Positive DIR level
M1 / X-left	9	8	22	26	HIGH
M2 / X-right	7	6	23	27	HIGH
M3 / Y	5	4	24	28	HIGH
M4 / Z	3	2	25	29	LOW

These are Mega digital pin numbers, copied from AF-434/gantry_control/gantry_control.ino at commit 873e2b6. The executable reference sets both X motors' positive-is-clockwise settings to true, even though parts of its README describe opposite rotation. This version follows that executable configuration. Positive X means AF-434's forward direction, positive Y means right, and positive Z means up. A negative step count reverses the relevant direction pins.

ENA LOW enables a driver and retains holding torque; ENA HIGH frees it. ALM HIGH means a fault and ALM LOW means normal. Each motor has its own PUL, DIR, ENA, and ALM pins. The old five-motor assumption and the old limit-switch pin assignments are gone. The AF-434 reference repository was used for reading only.

Where the main parts live

```
Arduino/AgriGantry/AgriGantry.ino  Complete Arduino firmware
  Settings near the top          Pins, directions, travel
  Motor control section          Timer1 pulses and velocity ramps
  Position section               Counts and manual references
  Alarm section                  Four driver alarms
  Command section                Commands and immediate O stop
Pi/gantry_keyboard.py            Hold-to-move keys on the computer
Pi/Pi_FarmBotController_Class.py  Typed commands on the computer
```

From a command to movement

What happens when you send X1000

The Python controller sends X1000 followed by a newline over USB serial at 115200 baud. CommandProcessor checks that X is a real axis and 1000 is a valid signed number. MotorControl adds 1000 to X's current position to get the target, checks the stop latch, and sets the X direction pins. It then generates 1000 step cycles. In each cycle it raises both X pulse pins before lowering either, so M1 and M2 each receive 1000 pulses. X's stored position increases by 1000, rather than 2000, because the two motors represent one axis. Y and Z receive no pulses during that command.

The motor engine now uses AF-434's Timer1 design: a 20,000 Hz interrupt generates pulses while a 1,000 Hz controller smoothly adjusts velocity. A pulse is held HIGH for the configured three microseconds. Default speed is 1000 steps/second, adjustable from 100 to 8000, and acceleration is 3000 steps/second squared. Release a key or press Space and velocity ramps down to zero; reversing first slows through zero, then accelerates the other way. At default speed a normal release takes roughly 0.33 seconds and 167 steps. For a finite move, remaining distance determines when braking begins; the final approach uses a low crawl speed and the pulse generator stops at the exact target count. Keyboard axes can run together; new finite position moves are accepted while idle.

What the position number means

Position is a software count of the pulses the Arduino issued. X1000 changes X by +1000; Y-500 changes Y by -500. The Arduino does not read the motors' encoder positions over the alarm wires. The integrated drivers handle their own internal feedback. If a motor is moved by hand, power is lost, or motion is interrupted abruptly, the software count may no longer describe the physical location. An emergency stop therefore invalidates the manual position references. Clearing a stop allows new relative moves, but does not recreate a physical zero.

No switches means a manual origin

Relative moves work immediately, without homing. To establish a physical reference, use small relative moves to reach a known origin, then send 0 to record all current axis positions as zero. Use 0X, 0Y, or 0Z to reference one axis. The command records the current location and produces no motion. H, HX, HY, HZ, and HALL only explain that homing is unavailable. Nothing searches for an end stop.

Percentage positioning also requires a known travel length. X_TRAVEL_STEPS, Y_TRAVEL_STEPS, and Z_TRAVEL_STEPS near the top of AgriGantry.ino start at 0 because the measured usable distances are unknown. Set an axis's actual travel in pulses, upload, then manually establish its origin with 0 or its per-axis version. PX50 can then move X to half of that travel. Referenced relative moves are clipped to the configured range of 0 through travel. Until both the reference and travel exist, there are no physical software bounds; percentage moves are refused.

Keyboard movement and stopping

Hold a key, release to slow down

Pi/gantry_keyboard.py starts a global pynput listener. Hold W/S to move X, A/D for Y, and Q/E for Z. Releasing a key ramps that axis to rest, while other held axes keep moving. If opposite keys on one axis are held, the latest press wins; releasing it falls back to the other held key. Space requests a smooth stop on all axes. The controller sends a heartbeat every 100 milliseconds. If the Arduino receives no jog heartbeat for 500 milliseconds, it slows jogging to a stop. Typed finite moves finish without requiring heartbeats.

O is the immediate emergency stop: it cancels every velocity and target instead of decelerating. It also works during an incomplete or overflowing serial command. All movement stays blocked until C in keyboard mode, or CLEAR/S0 in typed mode. A clear request succeeds only when the live motor alarms are normal. Release and press movement keys again after clearing; old held keys never restart by themselves. Drivers retain holding torque, including Z. This is a software stop requiring a functioning controller, USB connection, and Arduino.

What happens when a motor reports an alarm

ALM pins 26, 27, 28, and 29 are monitored continuously. A HIGH input sustained for five milliseconds latches the same global stop as O and identifies the motor. Shorter transients do not latch. Returning LOW does not resume movement; clear the stop explicitly after resolving the fault. This uses AF-434’s actual active-HIGH alarm behavior.

Keyboard controls

```
W / S           X forward / backward, both motors
A / D           Y left / right
Q / E           Z up / down
Release key     Smooth stop on that axis
Space           Smooth stop on ALL axes
O               Immediate latched EMERGENCY STOP
C               Clear stop after fixing any alarm
+ / -           Increase / decrease jog speed
P               Show positions and velocities
0               Record manual zero while stopped
Esc             Smooth stop and exit
```

The listener captures keys even while another window has focus, so close keyboard control before typing elsewhere. Run one controller at a time. The typed interface still accepts X1000, Y-500, PX25/PY50/PZ75, 0X/0Y/0Z, STATUS, ALM, and HELP. A Serial Monitor may require Send/Enter to transmit O; the Python listeners send it on the actual keypress.

Running and understanding it

Build, upload, and run

Open Arduino/AgriGantry/AgriGantry.ino in Arduino IDE. This one file contains all the gantry firmware. Select Arduino Mega or Mega 2560, choose the Mega's port, and click Upload. Close Serial Monitor, then run the Python keyboard controller on your computer using the commands below from the repository root. Replace COM9 with your actual port; omit --port for automatic detection.

```
python -m pip install -r Pi/requirements.txt
python Pi/gantry_keyboard.py --port COM9

Optional typed interface, after closing keyboard control:
python Pi/Pi_FarmBotController_Class.py --port COM9
```

A desktop session with keyboard access and the listed packages is required. Startup pulses DTR to reset the Mega, clears old serial transfers, waits for boot, and verifies the firmware identity. Startup motor alarms are displayed and block keyboard movement. This reset sequence recovered the COM9 idle write timeout. Use --list-ports to find the Mega if needed. Esc or Ctrl+C requests a smooth stop before closing; after three seconds without confirmation it sends an emergency stop. Startup holds torque and issues no movement pulses.

What was verified

The complete .ino compiled using Arduino IDE's build tool for the Mega 2560: 16,688 bytes of program storage and 548 bytes of static RAM. Native tests exercise the firmware against a simulated 20 kHz timer and pins, with 3,278 checks across unknown-travel and configured-travel builds. The same checks run directly against the single-file sketch with --standalone. They cover paired pulses, finite targets and braking, key-release slowing, reversal through zero, simultaneous axes, heartbeat loss, O stops, alarms, zero restrictions, and software travel bounds. Twenty-five Python tests cover key handling, stop states, startup reset and alarm preservation, serial errors, and firmware identification.

Movement was checked through build and simulation tests. Live USB communication on COM9 passed two consecutive connections, firmware/status queries, and 80 idle heartbeat writes using the new reset sequence. No movement commands were sent during that check. No firmware was uploaded by the agent and physical movement remains untested. The commands below repeat the automated checks; the native runner uses MSVC.

```
python Farm-Bot/test/run_native.py --standalone
python -m unittest discover -s Pi -p 'test_*.py'
python Farm-Bot/tools/make_guide.py
```